期中试卷讲解

填空题1、算法A、B、C的时间复杂度依次为O(n!)和$O(2^n)$和$O(n^{100})$，其中时间复杂度最高的是算法_____。

答：O(n!)

时间复杂度对比序列：

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

填空题2、单循环链表表示的队列长度为n，若只设尾指针，则进队的时间复杂度为_____。

答：O(1)

```
// 入队操作（仅核心步骤）
void enqueue(Node *&rear, int data) {
    Node *new_node = (Node *)malloc(sizeof(Node)); // 步骤1: 创建新节点
    new_node->data = data;
    new_node->next = rear->next; // 步骤2: 新节点next指向队头
    rear->next = new_node;       // 步骤3: 原队尾指向新节点
    rear = new_node;             // 步骤4: 更新尾指针
}
```

单循环链表表示的队列长度为n，若只设头指针，则进队的时间复杂度为 **O(n)**

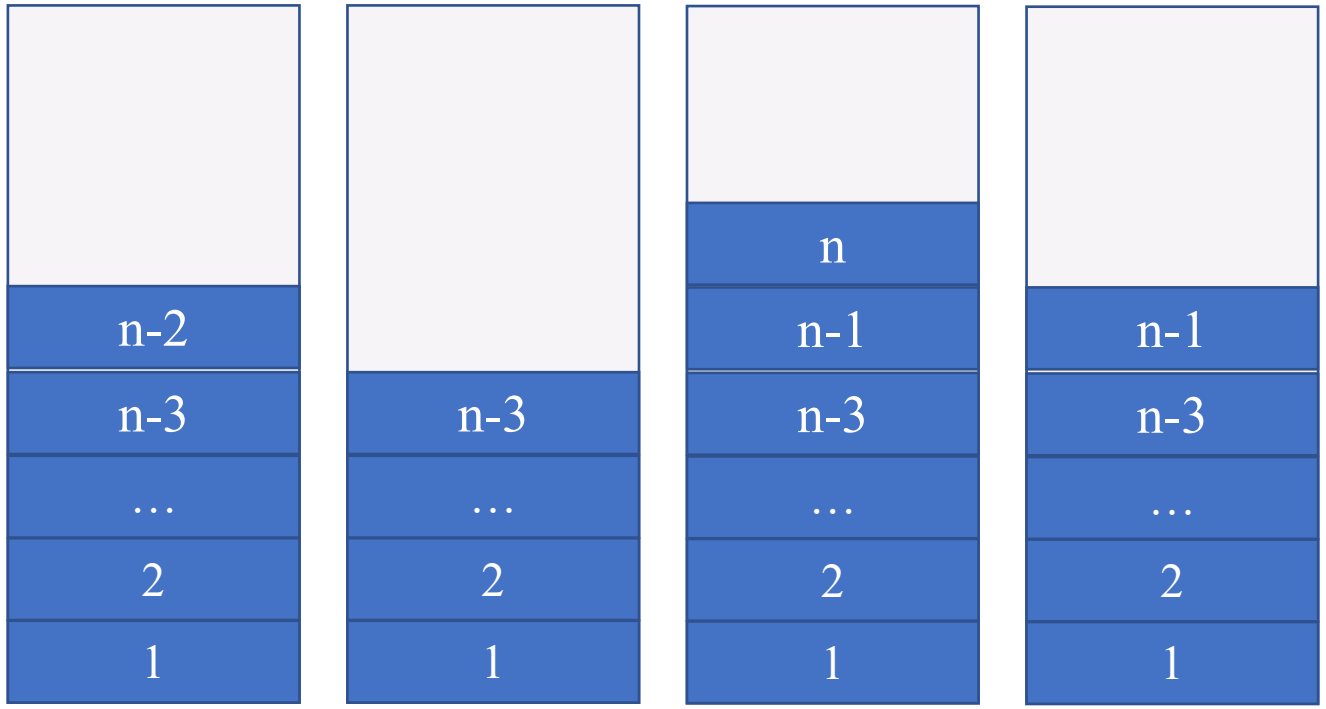
单循环链表表示的队列长度为n，若只设尾指针，则出队的时间复杂度为 **O(1)**

单循环链表表示的队列长度为n，若只设头指针，则出队的时间复杂度为 **O(1)**

填空题3、设元素$p_1p_2…p_n$依次进栈，出栈序列为$q_1q_2…q_n$，如果$q_1==p_{n-2}$，$q_2==p_n$，那么q_3为_____。

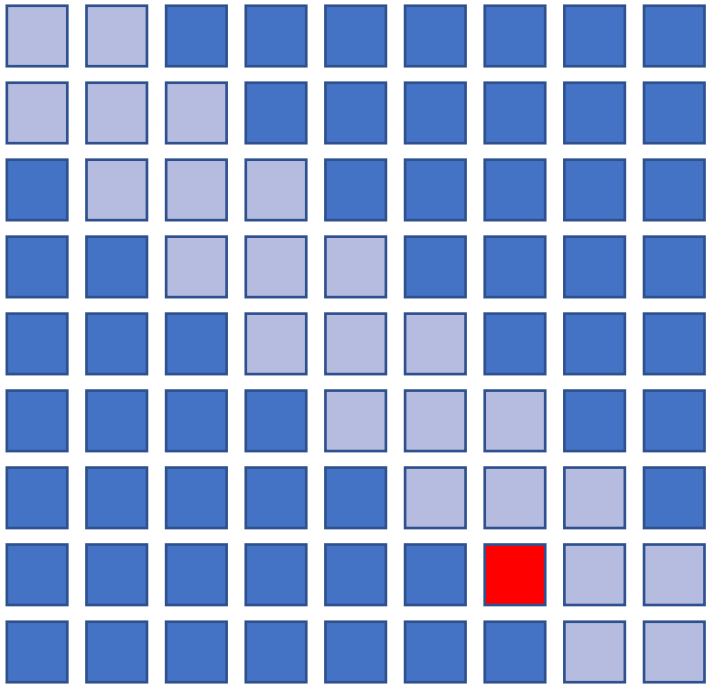
答：p_{n-1}

栈：先进后出



填空题4、已知三对角矩阵A[1..9, 1..9]的每个元素占用2个单元，现将其三条对角线上的元素逐行存储在起始地址为1000的连续内存单元中，则元素A[8][7]的存储地址为_____；如按列优先顺序进行存储，则A[8][7]的存储地址为_____。

答：(1) 1040；(2) 1038



$$1000 + 20 * 2 = 1040$$

$$1000 + 18 * 2 = 1030$$

填空题5、广义表A=((a), b, (c, d, (e, f)))的深度是_____，表尾是_____。

答：(1) 3；(2) (b, (c, d, (e, f)))

广义表是线性表的拓展，允许元素是原子，也可以是另一个广义表。

填空题6、一棵树x转换成二叉树y，x中有m个非叶结点，则y中没有右孩子的结点数目为_____。

答：m+1



m=3

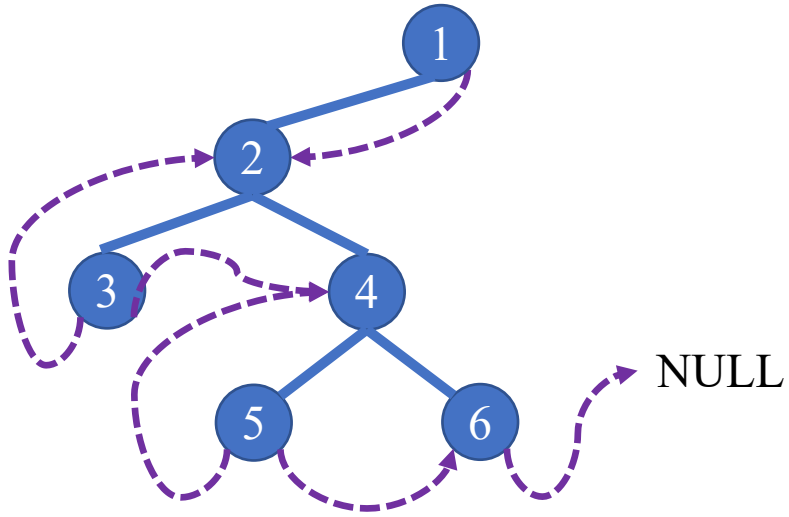
4

$$N - sum(t_i - 1) = N - (N - 1) - m = m + 1$$

填空题7、一棵左子树非空的二叉树在前序线索化后，其中的空链域个数为_____。

答：1

前序线索化的核心是：将二叉树的**空链域**替换为**线索**，指向节点的**前序前驱**或**前序后继**。
前序遍历：1 2 3 4 5 6



填空题8、试判断下面的关键码序列中哪一个是堆_____。

①{16, 23, 31,72, 94, 53} ②{94, 53, 31, 72, 16, 23} ③{16, 53, 23, 94, 31, 72}

④{16, 94, 23, 31, 53, 72} ⑤{94, 31,53, 23, 16, 72}

答：①

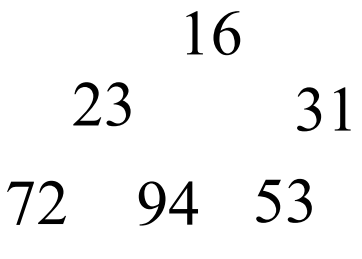
堆必须是一个完全二叉树

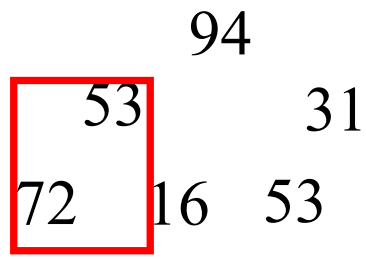
- **大顶堆（最大堆）**

  每个父节点的关键字 **大于等于** 其左右孩子的关键字。

  数学描述：`arr[parent] ≥ arr[left]` 且 `arr[parent] ≥ arr[right]`。

  特性：堆顶（根节点）是整个堆的**最大值**。



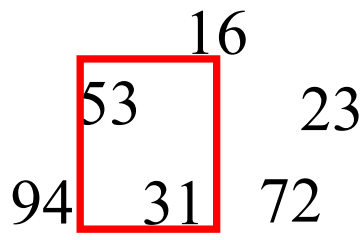


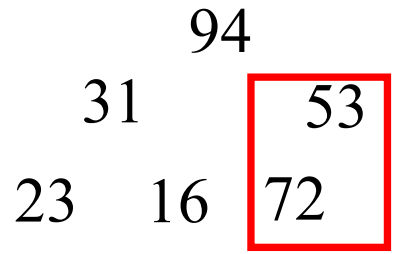
- **小顶堆（最小堆）**

  每个父节点的关键字 **小于等于** 其左右孩子的关键字。

  数学描述：`arr[parent] ≤ arr[left]` 且 `arr[parent] ≤ arr[right]`。

  特性：堆顶（根节点）是整个堆的**最小值**。





问答题1、某算法时间复杂度的递推关系如下，其中 n 为求解问题的规模，设 n 是 6 的正整数幂。请给出该算法时间复杂度的大 O 表示法及推导过程。

$$T(n)=\begin{cases}1 & (n=1)\\ 5T\left(\frac{n}{6}\right)+n & (n>1)\end{cases}$$

- $T(n)=\ 5T\left(\frac{n}{6}\right)+n=5\left(5T\left(\frac{n}{36}\right)+\frac{n}{6}\right)+n=\cdots$

  $=5^{\log_6 n}+5^{\log_6 n-1}\times 6+5^{\log_6 n-2}\times 6^2+\cdots+5\times\frac{n}{6}+n$ ①
- $T(n)\times\frac{5}{6}=\frac{5}{6}\times5^{\log_6 n}+5^{\log_6 n}+5^{\log_6 n-1}\times6\ +\ \ldots+5\times\frac{n}{6}$ ②
- $T(n)\times\frac{5}{6}=\frac{5}{6}\times5^{\log_6 n}+T(n)\ -\ n$ ③

- $T(n)\ =\ 6n\ -5^{\log_6 n+1}$

- 时间复杂度为O(n)

问答题1、某算法时间复杂度的递推关系如下，其中 n 为求解问题的规模，设 n 是 6 的正整数幂。请给出该算法时间复杂度的大 O 表示法及推导过程。

$$T(n)=\begin{cases}1 & (n=1)\\ 5T\left(\frac{n}{6}\right)+n & (n>1)\end{cases}$$

- 方法二：主定理
- 递归形式 $T(n)=aT\left(\frac{n}{b}\right)+f(n)$：$a=5, b=6, f(n)=n.$
- 计算 $n^{\log_b a}=n^{\log_6 5}$且 $\log_6 5\approx 0.898242.$
- 比较 $f(n)=n$ 与 $n^{\log_6 5}$：
- $f(n)=n=n^{\log_6 5+\varepsilon}$，其中 $\varepsilon=1-\log_6 5\approx 0.101758>0$。因此 $f(n)$ 多项式级别严格大于 $n^{\log_b a}$。
- 主定理的第三种情况适用 $f(n)=\Omega\left(n^{\log_b a+\varepsilon}\right)$，并需满足正则性条件：

$$a\cdot f\left(\frac{n}{b}\right)\le c\cdot f(n)\ \text{对于某个常数 c}\ <\ 1\ \text{且足够大的 n}.$$

- 验证：$a\cdot f\left(\frac{n}{b}\right)=5\cdot\frac{n}{6}=\frac{5}{6}n$, 取 $(c=\frac{5}{6}<1)$，正则性成立。
- 因此主定理给出 $T(n)=\Theta\left(f(n)\right)=\Theta(n).$

问答题2、已知模式串 P =“abababaab”，请写出按照KMP算法的要求，从next[0] = -1开始，依次推算出 next[1], next[2], …, next[6] 的过程。

求next[j]: 匹配+1，失配回溯，谁与谁匹配：下标j-1与next[j-1]

p[j-1]==p[next[j-1]] → next[j] = next[j-1] + 1

p[j-1]!=p[next[j-1]] 回溯到哪：next[next[j-1]]

- p[j-1]==p[next[next[j-1]]] → next[j] = next[next[j-1]] + 1
- 再失配：next[next[next[j-1]]]

问答题2、已知模式串 P =“abababaab”，请写出按照KMP算法的要求，从next[0] = -1开始，依次推算出 next[1], next[2], …, next[6] 的过程。

next[0]=-1

next[1]=0

比较下标 0 与 next[0]=-1

next[2]=0

比较下标 1 与 next[1]=0

P[1]=b ≠ P[0]=a

回溯：next[next[1]]=next[0]=-1

next[3]=1

比较下标 2 与 next[2]=0

P[2]=a ≠ P[0]=a

next[3]=next[2]+1

next[4]=2

比较下标 3 与 next[3]=1

P[3]=b = P[1]=b

next[4]=next[3]+1

next[5]=3

next[6]=1

比较下标 5 与 next[5]=3

回溯：next[next[5]]=next[3]=1

回溯：next[next[3]]=next[1]=0

问答题3、在一棵二叉树的前序遍历序列、中序遍历序列、后序遍历序列和层次序遍历序列中，任意两种序列的组合可以唯一地确定这棵二叉树吗?若可以,有哪些组合,若不可以,有哪些组合。请说明理由。

仔细读题！！！
有三个小问很多同学都没有看清。

标准答案：
不可以。只有前序 + 中序、中序 + 后序、中序 + 层次序（或者任意两个带中序的遍历）可以唯一确定。
只有这些序列的每一步的根结点位置和左右子树结点集合都是惟一确定的，于是整棵树被唯一确定。

为什么“带中序的三种”可以唯一确定？
1.需要知道谁是根节点。
2.知道根在中序序列中位置，就能把中序序列划成“左子树 + 右子树”，然后递归

中序遍历序列的特点：
在中序序列里，根的左边全是左子树结点，右边全是右子树结点。

问答题3、在一棵二叉树的前序遍历序列、中序遍历序列、后序遍历序列和层次序遍历序列中，任意两种序列的组合可以唯一地确定这棵二叉树吗?若可以,有哪些组合,若不可以,有哪些组合。请说明理由。

可行情况分析

以前序 + 中序为例（后序 + 中序同理）：

前序：根 左子树 右子树

⇒ 前序序列的第一个元素就是根。

在中序序列中找到这个根，把中序分成左、右两段：

左段 = 左子树的全部结点

右段 = 右子树的全部结点

然后在前序序列里，根后面那一段，按长度切给左子树和右子树，再递归重建。

因为每一步：

根是谁是唯一的（前序给出）；

左/右子树各有哪些结点是唯一的（中序给出）；

所以整棵树就被唯一确定。

中序 + 层次（层序）

这组略陌生一点，但原理一样：

层序：按从上到下、从左到右依次访问。

⇒ 层序序列的第一个元素就是根。

在中序中找到根，左边 = 左子树结点集合 L，右边 = 右子树结点集合 R

然后处理余下的层序序列（去掉根）：

把所有属于 L的结点，按在层序里出现的顺序取出来，得到“左子树的层序序列”；

把所有属于 R的结点，按在层序里出现的顺序取出来，得到“右子树的层序序列”。

于是，左右子树各自又有：

中序序列（刚才分出来的左/右段）；

层序序列（刚刚过滤出来的）。

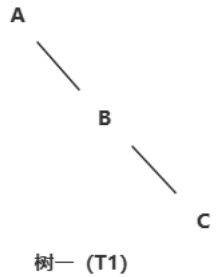
递归下去，每一步根是谁、中序如何切、层序如何分，全都确定，没有任何随意性，所以也唯一。

问答题3、在一棵二叉树的前序遍历序列、中序遍历序列、后序遍历序列和层次序遍历序列中，任意两种序列的组合可以唯一地确定这棵二叉树吗?若可以,有哪些组合,若不可以,有哪些组合。请说明理由。

为什么“没有中序的三种组合”不能唯一确定？

关键缺什么？
缺少了“左子树结点集合和右子树结点集合的划分信息”。
在不包含中序的三种组合里，对于“只有一个孩子”的结点，你无法判断这个孩子是“左孩子”还是“右孩子”，而这些不同的选择会保持某些遍历不变，从而导致不同的二叉树有相同的两种遍历序列。

举一个反例：

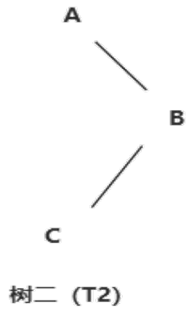


树一（T1）

前序（根-左-右）：
A B C

后序（左-右-根）：
C B A

层序（按层）：
A B C



树二（T2）

前序（根-左-右）：
A B C

后序（左-右-根）：
C B A

层序（按层）：
A B C

可以看出来对于这两棵树来说他们的前序、后序和层序遍历都是一样的。

问答题4、一棵二叉树的数组存储表示为[a, b, c, d, #, e, f, #, g, #, #, h, #]，其中#表示空结点，请画出这棵二叉树的图形表示，并且加上中序线索。

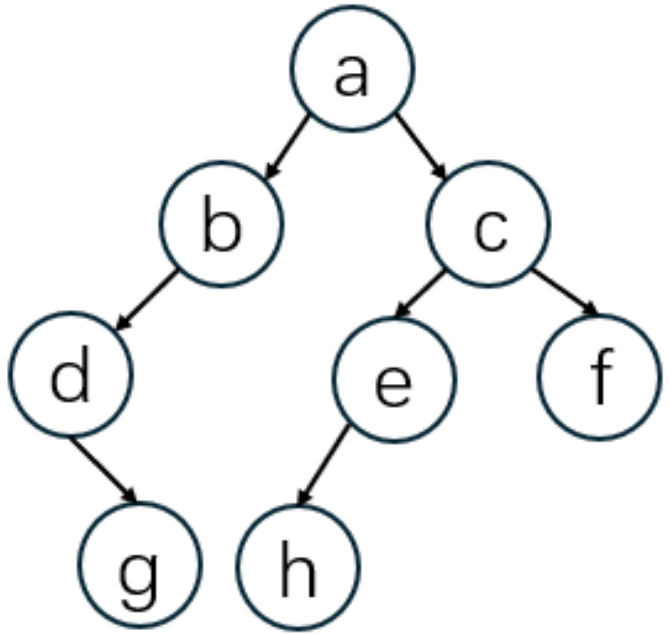
基本解题思想：

1. 二叉树的数组表示形式：将所有节点按照层序遍历的顺序存储在一个数组中，则每个节点都对应唯一的数组索引。根据层序遍历的特性，我们可以推导出父节点索引与子节点索引之间的“映射公式”：**若某节点的索引为 i，则该节点的左子节点索引为 2i+1，右子节点索引为 2i+2。**

答：中序遍历：

d -> g -> b -> a -> h -> e -> c -> f

0	1	2	3	4	5	6	7	8	9	10	11	12
a	b	c	d	#	e	f	#	g	#	#	h	#



问答题5、用于通信的电文由8个字符组成，字符在电文中出现的频率分别为0.09, 0.19, 0.02, 0.06, 0.30, 0.03, 0.21, 0.10。试为这8个字符设计哈夫曼编码。使用0~7的3位等长二进制表示形式是另一种编码方案，对于上述实例比较两种方案的优缺点。

主要错误点

1.哈夫曼树不是10、110、1110、11110……这样编码；编码只在叶子节点

2.计算过程中注意每次是最小两个树合并；

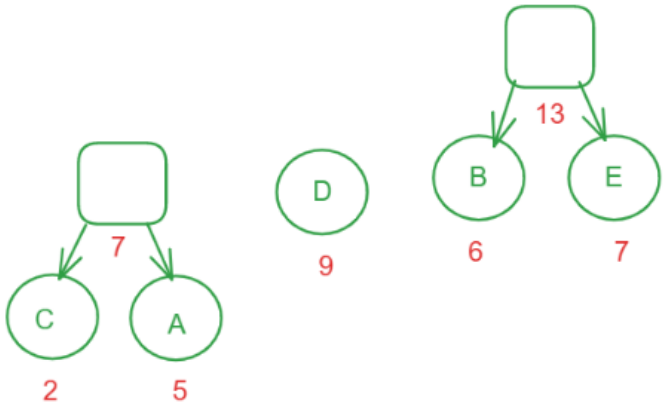
3.存储空间小，节省传输带宽，符合分布情况下带全路径长度是哈夫曼树最主要的优点

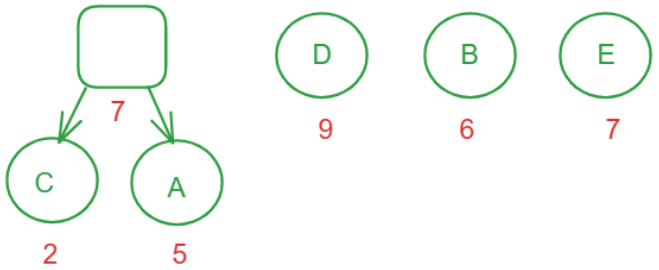
4.题目中有八个节点

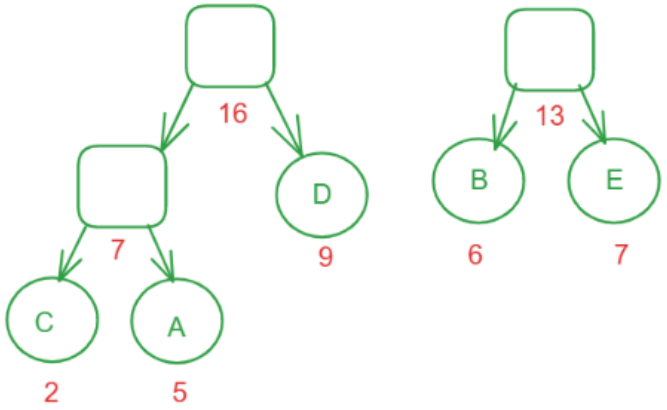
问答题5、用于通信的电文由8个字符组成，字符在电文中出现的频率分别为0.09, 0.19, 0.02, 0.06, 0.30, 0.03, 0.21, 0.10。试为这8个字符设计哈夫曼编码。使用0~7的3位等长二进制表示形式是另一种编码方案，对于上述实例比较两种方案的优缺点。

哈夫曼







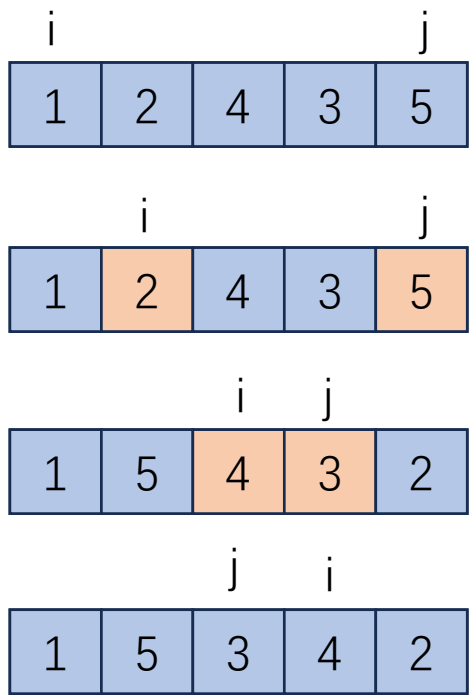


算法题1、一个数组中存放着未经排序的一组正整数，设计算法调整数组中的数据的顺序，使得所有奇数存放在所有偶数之前。要求尽量少的辅助空间和尽量快的速度。

要求：(1) 描述算法的基本思想并分析时间和空间复杂度；(2) 给出程序代码。

基本思路： （1）双指针一遍扫描，用类似于快排的交换过程。时间复杂度O(n)，空间复杂度O(1)

```
void move(int A[ ], int n)
{
    int i=0, j=n-1, tmp;
    while (i<j) {
        while (i<j&&A[i]%2==1)  i++;
        while (i<j&&A[i]%2==0)  j--;
        if (i<j) {
            tmp=A[i];  A[i]=A[j];  A[j]=tmp;
            i++;  j--;
        }
    }
}
```



算法题1、一个数组中存放着未经排序的一组正整数，设计算法调整数组中的数据的顺序，使得所有奇数存放在所有偶数之前。要求尽量少的辅助空间和尽量快的速度。

错误类型：

（1）没有分析时间复杂度、空间复杂度
（2）没有写代码思路
（3）交换时没有判断 i, j 是否合法
（3）使用辅助空间来做

```
void move(int A[ ], int n)
{
    int i=0, j=n-1, tmp;
    while (i<j) {
        while (i<j&&A[i]%2==1)  i++;
        while (i<j&&A[i]%2==0)  j--;
        if (i<j) {
            tmp=A[i];  A[i]=A[j];  A[j]=tmp;
            i++;  j--;
        }
    }
}
```

算法题2、分析以下递归函数print(**int** w)，回答：(1) 调用语句print(4)的结果是什么？(2) 不用栈将该递归算法改为非递归。

```
void print(int w) {
        int i;
        if (w != 0) {
                print(w-1);
                for (i = 1; i <= w; i++)
                        cout << w;
                cout << endl;
        }
}
```

函数 print(w) 的核心是：

- 递归调用 print(w-1)，先处理比 w 小的数字。
- 打印 w 个数字 w，每次打印完成后换行。

递归的终止条件是 w == 0，此时不做任何操作。

所以，递归的行为是：

- 先从 1 到 w-1 打印（通过递归实现）
- 再打印 w 个 w

递归顺序是从 1 到 w打印行

实际上，print(w) 等价于：

```
for (int j = 1; j <= w; j++) {
   // 打印 j 个 j
}
```

可以直接用两个嵌套循环实现

算法题2、分析以下递归函数print(**int** w)，回答：(1) 调用语句print(4)的结果是什么？(2) 不用栈将该递归算法改为非递归。

```
void print(int w) {
        int i;
        if (w != 0) {
                print(w-1);
                for (i = 1; i <= w; i++)
                        cout << w;
                cout << endl;
        }
}
```

参考答案

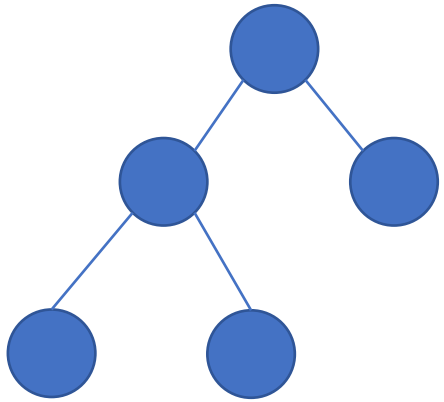
print(4)的结果是：

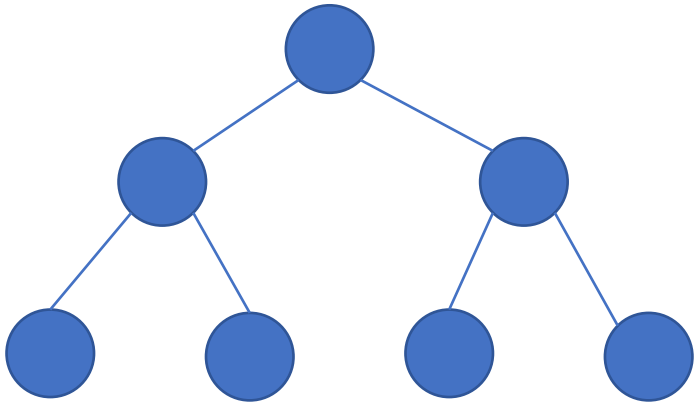
```
        1
        2  2
        3  3  3
        4  4  4  4
```

```
void print(int w) {
    int k = 1;
    while (k != w+1)
    {
      for (int i = 1; i <=k; i++)
        cout << k;
      cout << endl;
      k++;
    }
}
```

算法题3、设计一个C++函数DescendLeavies(BiTree *T)，输出二叉树中距离**给定结点T最近（经过的分支数最少）的T的叶子子孙结点。**同时，**分析该算法的时间复杂度。**

层次遍历碰到的第一个叶子结点。





```
BiTree* DescendLeavies(BiTree* T) {
    InitQueue(Q);
    EnQueue(Q, T);

    while (!QueueEmpty(Q)) {
        DeQueue(Q, p);

        if (p->lchild == null && p->rchild == null)
            return p;

        if (p->lchild) EnQueue(Q, p->lchild);
        if (p->rchild) EnQueue(Q, p->rchild);
    }
    return null;
}
```

算法题3、设计一个C++函数DescendLeavies(BiTree *T)，输出二叉树中距离**给定结点T最近（经过的分支数最少）的T的叶子子孙结点。**同时，**分析该算法的时间复杂度。**

```
void dfsFindNearestLeaf(BiTree* node, int depth, int&
minDist, BiTree*& bestLeaf) {
    if (node == nullptr) return;

    // 如果是叶子：比较并更新最优答案
    if (node->lchild == nullptr && node->rchild == nullptr) {
        if (bestLeaf == nullptr || depth < minDist) {
            minDist = depth;
            bestLeaf = node;
        }
        return;
    }

    // 继续往左右子树找（深度 +1）
    dfsFindNearestLeaf(node->lchild, depth + 1, minDist,
bestLeaf);
    dfsFindNearestLeaf(node->rchild, depth + 1, minDist,
bestLeaf);
}
```

```
BiTree* DescendLeaves_DFS(BiTree* T, int&
minDist) {
    BiTree* bestLeaf = nullptr;
    minDist = 1000000000; // 给一个很大的初值
即可

    dfsFindNearestLeaf(T, 0, minDist, bestLeaf);
    return bestLeaf;
}
```

算法题3、设计一个C++函数DescendLeavies(BiTree *T)，输出二叉树中距离**给定结点T最近（经过的分支数最少）的T的叶子子孙结点。**同时，**分析该算法的时间复杂度。**

```
BiTree* DescendLeaves_DFS_Iter(BiTree* T, int& minDist) {
    if (T == nullptr) {
        minDist = 0;
        return nullptr;
    }

    struct Frame {
        BiTree* node;
        int depth;   // 从 T 到 node 的边数
    };

    stack<Frame> st;
    st.push({T, 0});

    BiTree* bestLeaf = nullptr;
    minDist = 1000000000; // 足够大的初值

    while (!st.empty()) {
        Frame cur = st.top();
        st.pop();
        BiTree* node = cur.node;
        int depth = cur.depth;

        if (node != nullptr) {
            // 如果是叶子：比较并更新
            if (node->lchild == nullptr && node->rchild == nullptr) {
                if (bestLeaf == nullptr || depth < minDist) {
                    minDist = depth;
                    bestLeaf = node;
                }
            }
            // 否则：把孩子压栈继续走
            else {
                if (node->rchild) st.push({node->rchild, depth + 1});
                if (node->lchild) st.push({node->lchild, depth + 1});
            }
        }
    }
    return bestLeaf;
}
```